Imports

```
In [1]:
# General util imports
import os

# Typing imports
from typing import Union

from dotenv import load_dotenv
import kagglehub
import shutil
from pathlib import Path
import tkinter as tk
from tkinter import filedialog

# Math and ML imports
import pandas as pd
import torch
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.amp import autocast, GradScaler
import torchvision
from torchvision.models.detection import FasterRCNN
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchmetrics.detection.mean_ap import MeanAveragePrecision
from sklearn.model_selection import train_test_split
import albumentations as A
from albumentations.pytorch import ToTensorV2
import cv2
import numpy as np

# Visualization imports
from matplotlib import pyplot as plt
from matplotlib import patches
from matplotlib.lines import Line2D
from tqdm import tqdm
```

Custom Util functions

```
In [2]:
def merge_csvs(files: list[str], output: str) -> None:
    """
    Merges multiple CSV files into a single output file. Each input file is read
    and concatenated, and the combined output is saved into the specified output
    file. The operation only proceeds if two or more input files are provided.

    :param files: A list of file paths to the input CSV files to be merged.
    :param output: The file path where the merged CSV content should be saved.
    :return: None
    """

    # Do nothing if 0 or 1 files are provided
    if len(files) < 2:
        return
    dfs = [pd.read_csv(f) for f in files]
    pd.concat(dfs).to_csv(output, index=False)
```

Kaggle setup and dataset download

```
In [3]:
# Get Kaggle credentials from .env
load_dotenv()
kaggle_username, kaggle_key = os.getenv("KAGGLE_USERNAME"), os.getenv("KAGGLE_KEY")

# Set credentials in os.environ so Kaggle API can find them automatically
if kaggle_username:
    os.environ["KAGGLE_USERNAME"] = kaggle_username
if kaggle_key:
    os.environ["KAGGLE_KEY"] = kaggle_key

if not kaggle_key:
    print("Warning: KAGGLE_KEY not found. Please set it as an environment variable.")
```

```
In [4]:
# Download dataset from Kaggle if not already downloaded
path = kagglehub.dataset_download("awsaf49/cbis-ddsm-breast-cancer-image-dataset")

print("Path to dataset files:", path)
```

```
Path to dataset files: C:\Users\nerfb\.cache\kagglehub\datasets\awsaf49\cbis-ddsm-breast-cancer-image-dataset\versions\1
```

Move data to local project

```
In [5]:
source_dir = Path(path)
target_dir = Path('../data/cbis-ddsm')

# Ensure the target directory exists and is empty
if not os.path.exists(target_dir):
    os.makedirs(target_dir)
if len(os.listdir(target_dir)) == 0:
    shutil.copytree(str(source_dir), str(target_dir),
                    dirs_exist_ok=True)  # Copy the data to the target directory instead of moving it
    print("Copied data to target location")

    # Merge Mass CSVs into a single file
    merge_csvs(['../data/cbis-ddsm/csv/mass_case_description_test_set.csv',
                '../data/cbis-ddsm/csv/mass_case_description_train_set.csv'],
               '../data/cbis-ddsm/csv/mass_case_description_merged.csv')
else:
    print("Target directory is not empty. Skipping data copy.")
```

```
Target directory is not empty. Skipping data copy.
```

Create dataframes

```
In [6]:
# Full dataframe
df = pd.read_csv('../data/cbis-ddsm/csv/dicom_info.csv')

# Change image path to relative path
df['image_path'] = df['image_path'].apply(lambda x: x.replace('CBIS-DDSM/', '../data/cbis-ddsm/'))

# Remove calcification images. Only focusing on masses for now
df = df[~df['PatientID'].str.contains('Calc')]

df.reset_index(inplace=True, drop=True)

# ROI mask dataframe
roi_masks_df = df[df['SeriesDescription'] == 'ROI mask images']

# Full mammogram dataframe
full_mammograms_df = df[df['SeriesDescription'] == 'full mammogram images']

print(f"Total ROIs: {len(roi_masks_df)}")
print(roi_masks_df['image_path'].head(5))
```

```
Total ROIs: 1696
5     ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
10    ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
12    ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
18    ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
19    ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
Name: image_path, dtype: str
```

Pair ROIs with full images

```
In [7]:
# Extract the base patient ID from each dataframe
roi_masks_df = roi_masks_df.copy()
roi_masks_df['base_id'] = roi_masks_df['PatientID'].str.extract(r'(.*_[A-Z]+_[A-Z]+)')

full_mammograms_df = full_mammograms_df.copy()
full_mammograms_df['base_id'] = full_mammograms_df['PatientID']

# Merge the two dataframes on the base_id column
merged_df = pd.merge(
    full_mammograms_df,
    roi_masks_df,
    on='base_id',
    suffixes=('_full', '_roi')
)

print(f"Successfully paired {len(merged_df)} mammogram/ROI pairs.")

initial_len = len(merged_df)
# Remove entries where the dimensions of the full image and ROI don't match
merged_df = merged_df[
    (merged_df['Rows_full'] == merged_df['Rows_roi']) &
    (merged_df['Columns_full'] == merged_df['Columns_roi'])
    ]
print(f"Removed {initial_len - len(merged_df)} entries due to mismatched dimensions.")

# Group ROIs of the same image together
paired_df = merged_df.groupby('image_path_full').agg({
    'image_path_roi': list,
    'PatientID_full': 'first',
    'base_id': 'first'
}).reset_index()

print(f"Consolidated into {len(paired_df)} image entries.")
```

```
Successfully paired 1696 mammogram/ROI pairs.
Removed 78 entries due to mismatched dimensions.
Consolidated into 1514 image entries.
```

```
In [8]:
paired_df.head(5)
```

Out[8]:

	image_path_full	image_path_roi	PatientID_full	base_id
0	../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....	[../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...	Mass-Test_P_00576_LEFT_MLO	Mass-Test_P_00576_LEFT_MLO
1	../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....	[../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...	Mass-Test_P_01510_RIGHT_MLO	Mass-Test_P_01510_RIGHT_MLO
2	../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....	[../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...	Mass-Test_P_01294_RIGHT_MLO	Mass-Test_P_01294_RIGHT_MLO
3	../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....	[../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...	Mass-Training_P_00281_LEFT_MLO	Mass-Training_P_00281_LEFT_MLO
4	../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....	[../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...	Mass-Training_P_00106_RIGHT_MLO	Mass-Training_P_00106_RIGHT_MLO

Check dataframe sizes

```
In [9]:
print("Original df: ", df.shape[0])

print("Mammograms: ", full_mammograms_df.shape[0])
print("ROI masks: ", roi_masks_df.shape[0])

print("Pairs: ", merged_df.shape[0])
print("Entries: ", paired_df.shape[0])
```

```
Original df:  5550
Mammograms:  1592
ROI masks:  1696
Pairs:  1618
Entries:  1514
```

Dataset Exploration

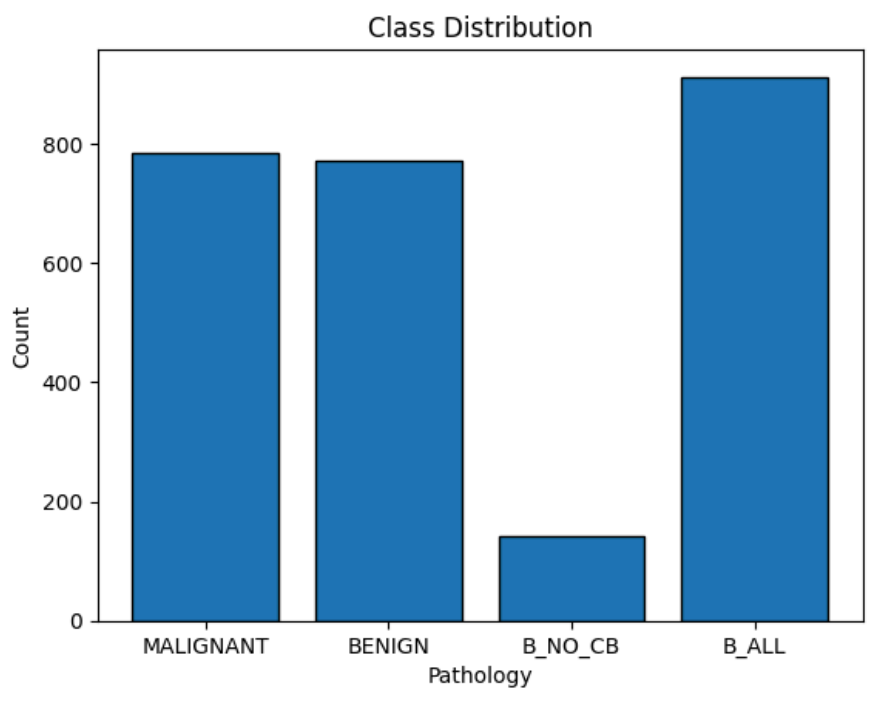
```
In [10]:
mass_info_df = pd.read_csv('../data/cbis-ddsm/csv/mass_case_description_merged.csv')
```

Class Distribution

```
In [11]:
class_counts = mass_info_df['pathology'].value_counts()

# Rename the BENIGN_WITHOUT_CALLBACK column to B_NO_CB to fir in class distribution graph
class_counts.rename({'BENIGN_WITHOUT_CALLBACK': 'B_NO_CB'}, axis=0, inplace=True)

# Combine the BENIGN and B_NO_CB columns into a single column named B_ALL
class_counts['B_ALL'] = class_counts['BENIGN'] + class_counts['B_NO_CB']

class_counts
```

```
Out[11]:
pathology
MALIGNANT    784
BENIGN       771
B_NO_CB      141
B_ALL        912
Name: count, dtype: int64
```

```
In [12]:
plt.bar(class_counts.index, class_counts.values, edgecolor='black')
plt.xlabel('Pathology')
plt.ylabel('Count')
plt.title('Class Distribution')
plt.show()
```



```
In [13]:
total_len = len(mass_info_df)

percent_malignant = class_counts['MALIGNANT'] / total_len
percent_benign = class_counts['B_ALL'] / total_len

print(f'Malignant: {percent_malignant:.2%} | Benign: {percent_benign:.2%}')
```

```
Malignant: 46.23% | Benign: 53.77%
```

Images with More than one ROI

```
In [14]:
# Get images that appear at least twice in the dataframe
duplicates = mass_info_df[mass_info_df.duplicated(subset=['image file path'], keep=False)].reset_index(drop=True)

duplicates
```

Out[14]:

	patient_id	breast_density	left or right breast	image view	abnormality id	abnormality type	mass shape	mass margins	assessment	pathology	subtlety	image file path	cropped image file pa
0	P_00116	2	RIGHT	CC	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	5	MALIGNANT	5	Mass-Test_P_00116_RIGHT_CC/1.3.6.1.4.1.9590.10...	Ma Test_P_00116_RIGHT_CC_1/1.3.6.1.4.1.959
1	P_00116	2	RIGHT	CC	2	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	5	MALIGNANT	5	Mass-Test_P_00116_RIGHT_CC/1.3.6.1.4.1.9590.10...	Ma Test_P_00116_RIGHT_CC_2/1.3.6.1.4.1.959
2	P_00116	2	RIGHT	MLO	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	5	MALIGNANT	5	Mass-Test_P_00116_RIGHT_MLO/1.3.6.1.4.1.9590.1...	Ma Test_P_00116_RIGHT_MLO_1/1.3.6.1.4.1.959
3	P_00116	2	RIGHT	MLO	2	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	5	MALIGNANT	5	Mass-Test_P_00116_RIGHT_MLO/1.3.6.1.4.1.9590.1...	Ma Test_P_00116_RIGHT_MLO_2/1.3.6.1.4.1.959
4	P_00173	2	RIGHT	CC	1	mass	OVAL	CIRCUMSCRIBED	3	BENIGN	5	Mass-Test_P_00173_RIGHT_CC/1.3.6.1.4.1.9590.10...	Ma Test_P_00173_RIGHT_CC_1/1.3.6.1.4.1.959
...	...	...	...	...	...	...	...	...	...	...	...	...	
170	P_01656	2	LEFT	CC	1	mass	OVAL	ILL_DEFINED	4	MALIGNANT	4	Mass-Training_P_01656_LEFT_CC/1.3.6.1.4.1.9590...	Ma Training_P_01656_LEFT_CC_1/1.3.6.1.4.1.9
171	P_01656	2	LEFT	CC	2	mass	IRREGULAR	ILL_DEFINED	5	MALIGNANT	4	Mass-Training_P_01656_LEFT_CC/1.3.6.1.4.1.9590...	Ma Training_P_01656_LEFT_CC_2/1.3.6.1.4.1.9
172	P_01656	2	LEFT	MLO	1	mass	OVAL	CIRCUMSCRIBED	5	MALIGNANT	5	Mass-Training_P_01656_LEFT_MLO/1.3.6.1.4.1.959...	Ma Training_P_01656_LEFT_MLO_1/1.3.6.1.4.1.
173	P_01656	2	LEFT	MLO	2	mass	LOBULATED	ILL_DEFINED	4	MALIGNANT	4	Mass-Training_P_01656_LEFT_MLO/1.3.6.1.4.1.959...	Ma Training_P_01656_LEFT_MLO_2/1.3.6.1.4.1.
174	P_01656	2	LEFT	MLO	3	mass	IRREGULAR	ILL_DEFINED	5	MALIGNANT	4	Mass-Training_P_01656_LEFT_MLO/1.3.6.1.4.1.959...	Ma Training_P_01656_LEFT_MLO_3/1.3.6.1.4.1.

175 rows × 14 columns

```
In [15]:
print(f'Found {duplicates['image file path'].nunique()} images with at least 2 ROI masks.')
```

```
Out[15]: Found 71 images with at least 2 ROI masks.
```

```
In [16]:
# Get the patient ID from the first duplicate image
pid = duplicates.iloc[0]['image file path']
pid = pid[:pid.index('/')]
pid
```

```
Out[16]: 'Mass-Test_P_00116_RIGHT_CC'
```

```
In [17]:
# Find the entry in the paired_df that corresponds to this patient ID
duplicate_idx = paired_df[paired_df['PatientID_full'] == pid].index[0]
paired_df.iloc[duplicate_idx]
```

```
Out[17]:
image_path_full    ../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1....
image_path_roi     [../data/cbis-ddsm/jpeg/1.3.6.1.4.1.9590.100.1...
PatientID_full                          Mass-Test_P_00116_RIGHT_CC
base_id                                 Mass-Test_P_00116_RIGHT_CC
Name: 360, dtype: object
```

Custom Dataset class

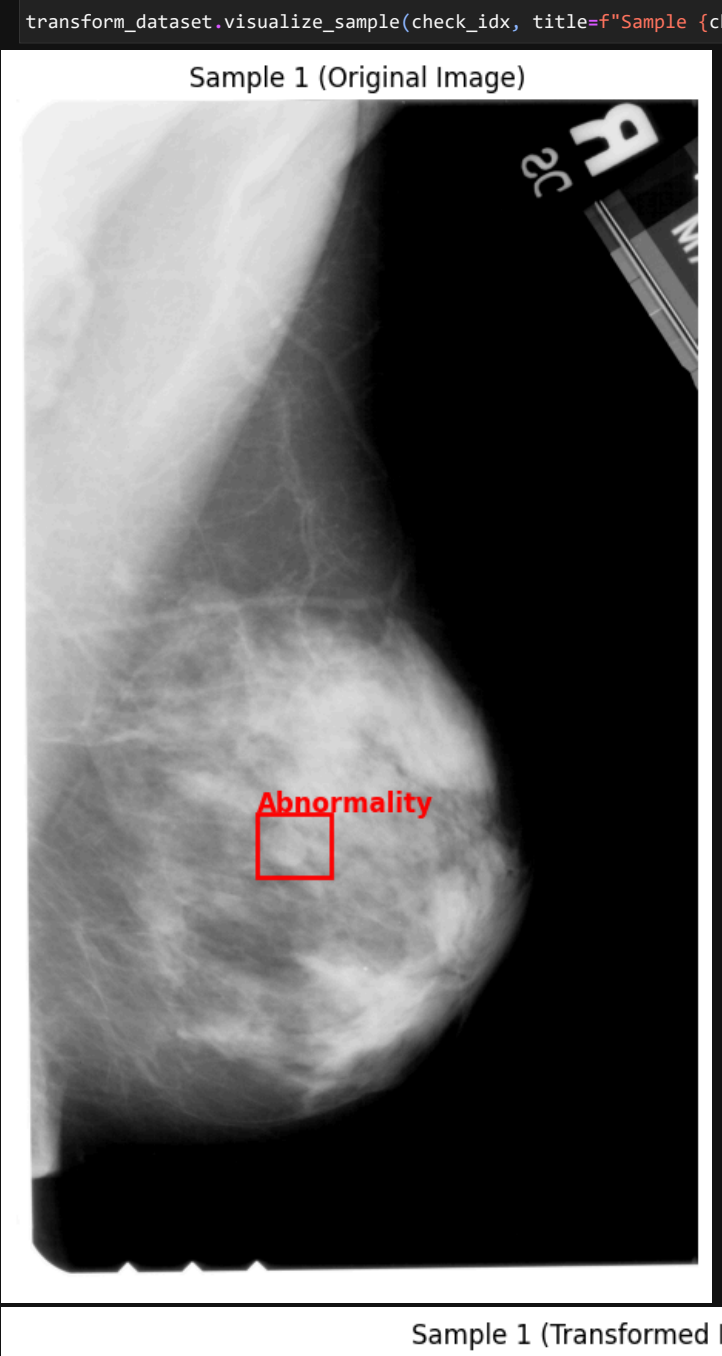
This class converts the ROI masks into bounding boxes for use in a PyTorch RCNN model.

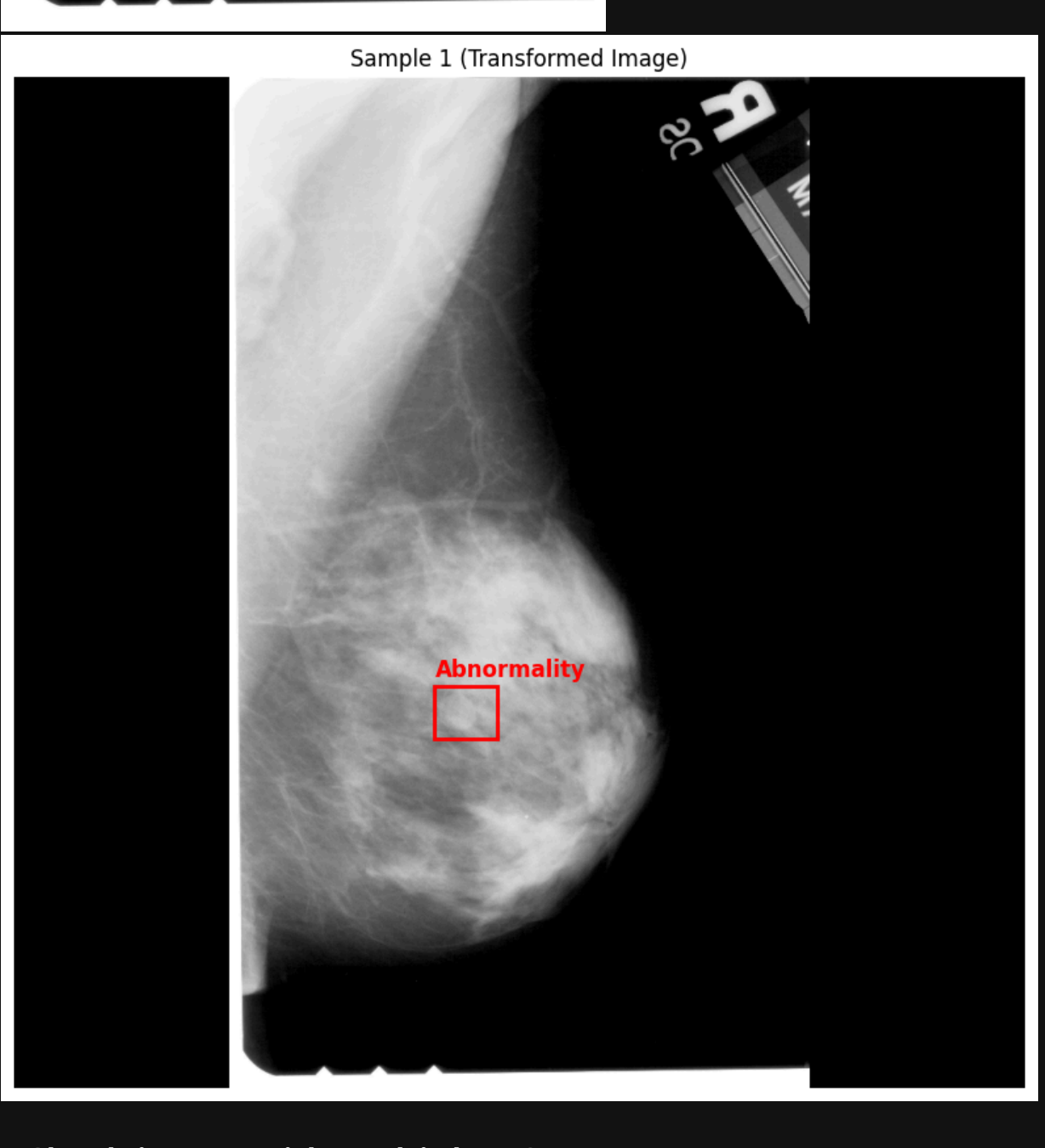
```
In [18]:
class CBISDDSM_RCNN_Dataset(Dataset):
    """
    Dataset class for managing CBIS-DDSM dataset configurations for object detection tasks.

    This class acts as a PyTorch Dataset for the CBIS-DDSM dataset, which is a collection
    of mammogram images annotated with region of interest (ROI) masks. The dataset is
    designed for object detection models such as Faster R-CNN. Each sample consists of an
    image and the corresponding bounding boxes (derived from ROI masks) along with labels.

    The class supports image transformations using libraries such as Albumentations,
    and provides a method to visualize images along with the annotated bounding boxes.

    :ivar df: Pandas dataframe containing metadata about the dataset. Each row corresponds
        to one mammogram image, and contains file paths to the image and associated
        ROI masks.
    :type df: pandas.DataFrame
    :ivar transform: Transformation pipeline for preprocessing the images and their bounding
        boxes. If None, no transformations will be applied.
    :type transform: callable or None
    """

    def __init__(self, dataframe: pd.DataFrame, transform: A.Compose | None = None):
        """
        Initializes the object with a dataframe and an optional transformation.

        This constructor creates an instance of the class with the provided
        dataframe. An optional transformation function can also be passed
        to modify or process the dataframe during further usage.

        :param dataframe: A pandas DataFrame containing the data to be
            processed.
        :type dataframe: pandas.DataFrame
        :param transform: An optional callable that takes the dataframe
            as input and returns a transformed dataframe. Defaults to None.
        :type transform: callable or None, optional
        """
        self.df = dataframe
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx: int) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
        """
        Retrieve an image tensor and its corresponding target dictionary containing bounding
        boxes and labels for regions of interest (ROIs) within the image.

        This method loads a mammogram image and its associated ROI masks. It calculates bounding
        boxes around each ROI, applies transformations if specified, and constructs a target dictionary
        suitable for training object detection models.

        :param idx: Index of the image and corresponding ROIs in the dataset.
        :type idx: int
        :return: A tuple containing the transformed tensor representation of the image and a
            target dictionary with bounding boxes, labels, and the image ID.
        :rtype: tuple[torch.Tensor, dict[str, torch.Tensor]]
        """

        # Get the image and ROI paths for the specified index
        img_path = self.df.iloc[idx]['image_path_full']
        roi_paths = self.df.iloc[idx]['image_path_roi']

        # Load the mammogram image and convert to RGB
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Load each ROI mask for the mammogram image
        boxes = []
        for r_path in roi_paths:
            # Read the mask and ensure it is the same size as the mammogram image
            mask = cv2.imread(r_path)
            mask = cv2.resize(mask, (img.shape[1], img.shape[0]))

            pos = np.where(mask > 0)

            # Convert to bounding box format
            if len(pos[0]) > 0:
                xmin, xmax = np.min(pos[1]), np.max(pos[1])
                ymin, ymax = np.min(pos[0]), np.max(pos[0])
                boxes.append([xmin, ymin, xmax, ymax])

        labels = [1] * len(boxes) if boxes else []

        # Transform the image and bounding boxes
        if self.transform:
            transformed = self.transform(image=img, bboxes=boxes, labels=labels)
            img_tensor = transformed['image']

            # Transform box and label tensors into the proper type
            if len(transformed['bboxes']) > 0:
                boxes = torch.as_tensor(transformed['bboxes'], dtype=torch.float32)
                labels = torch.as_tensor(transformed['labels'], dtype=torch.int64)
            else:
                boxes = torch.empty((0, 4), dtype=torch.float32)
                labels = torch.empty((0,), dtype=torch.int64)
        else:  # Default transformation
            img_tensor = torch.from_numpy(img).permute(2, 0, 1).float() / 255.0
            boxes = torch.as_tensor(boxes, dtype=torch.float32) if len(boxes) > 0 else torch.empty((0, 4),
                                                                                                  dtype=torch.float32)
            labels = torch.as_tensor(labels, dtype=torch.int64)

        # Create the target dictionary
        target = {
            "boxes": boxes,
            "labels": labels,
            "img_id": torch.tensor([idx]),
        }

        return img_tensor, target

    def visualize_sample(self, idx: int, title: str | None = None) -> None:
        """
        Visualize a sample from the dataset with its bounding box annotations.

        This method retrieves the image tensor and corresponding target annotations for
        the specified sample index. It then displays the image along with bounding boxes
        for each abnormality present in the target annotations. Bounding boxes are rendered
        as red rectangles with an accompanying label above each box. The title of the plot
        can be customized or defaults to "Sample Image and Bounding Box(es)" if not specified.

        :param idx: The index of the sample to retrieve and visualize.
        :type idx: int
        :param title: Optional title for the plot. Defaults to None, using the default title.
        :type title: str | None
        :return: None
        """

        # Get the image tensor and target annotations for the specified sample
        img_tensor, target = self[idx]

        try:
            # Image has multiple channels
            img_np = img_tensor.permute(1, 2, 0).numpy()
        except:
            # Image has only one channel
            img_np = img_tensor.squeeze(0).numpy()

        fig, ax = plt.subplots(1, figsize=(10, 10))
        ax.imshow(img_np, cmap='gray')

        # Extract bounding box(es) from the target annotations
        boxes = target["boxes"].numpy()
        for box in boxes:
            xmin, ymin, xmax, ymax = box
            width = xmax - xmin
            height = ymax - ymin

            rect = patches.Rectangle(
                (xmin, ymin),
                width,
                height,
                linewidth=2,
                edgecolor='r',
                facecolor='none'
            )

            ax.add_patch(rect)

            plt.text(xmin, ymin - 10, 'Abnormality', color='r', fontsize=12, weight='bold')

        plt.axis('off')
        plt.title("Sample Image and Bounding Box(es)" if not title else title)
        plt.show()
```

```
In [19]:
# Define the transform functions

# Resize image while keeping aspect ratio, Randomly flip images and alter brightness and contrast
rcnn_train_transform = A.Compose([
    A.LongestMaxSize(1_024),
    A.PadIfNeeded(1_024, 1_024),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.ToFloat(max_value=255.0),
    ToTensorV2()
],
    bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels'])
)

# Resize image while keeping aspect ratio
rcnn_test_transform = A.Compose([
    A.LongestMaxSize(1_024),
    A.PadIfNeeded(1_024, 1_024),
    A.ToFloat(max_value=255.0),
    ToTensorV2()
],
    bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels'])
)
```

Check the transformed images and bounding boxes for errors

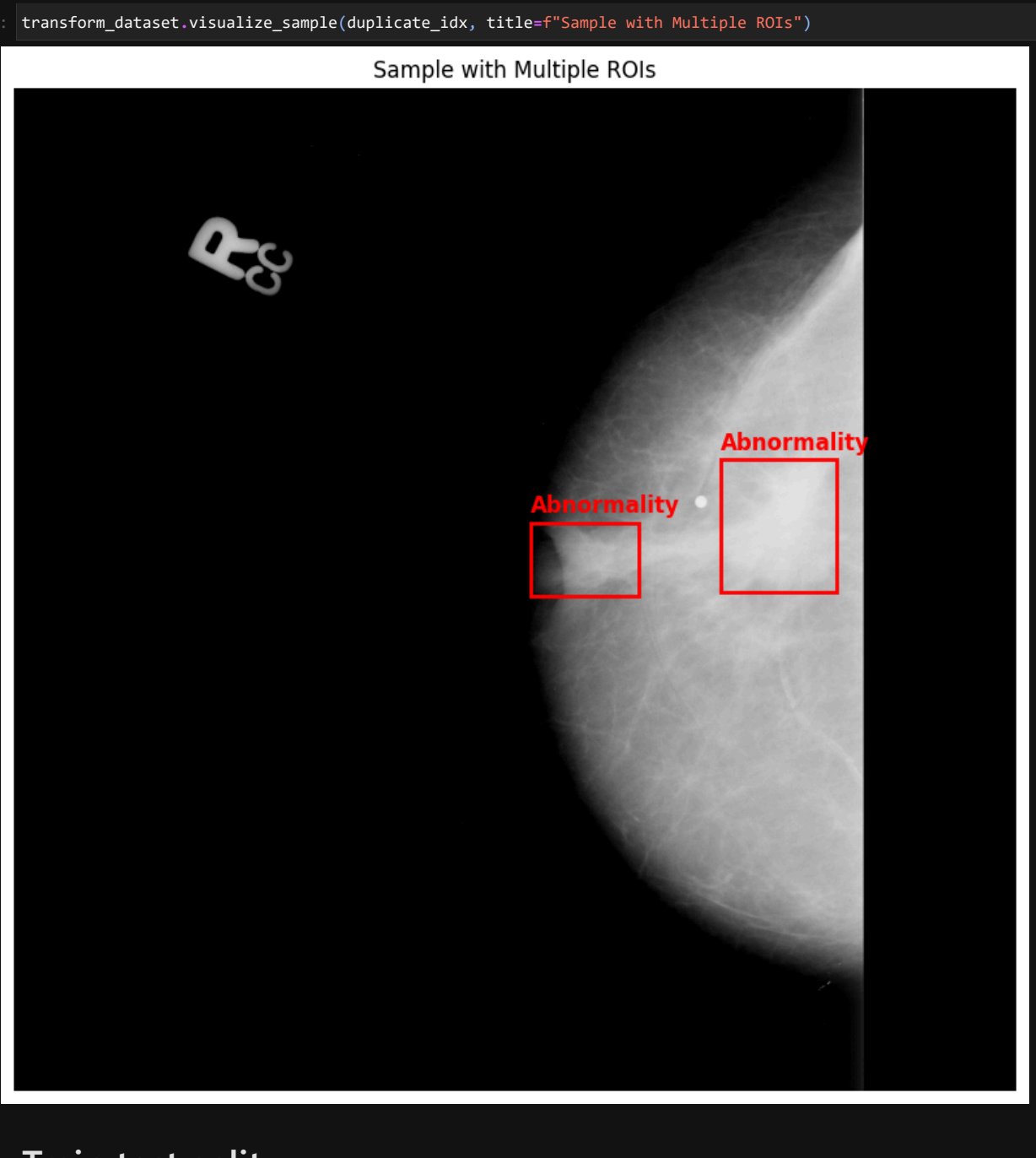
```
In [20]:
dataset = CBISDDSM_RCNN_Dataset(dataframe=paired_df)

transform_dataset = CBISDDSM_RCNN_Dataset(dataframe=paired_df, transform=rcnn_test_transform)

check_idx = 1

dataset.visualize_sample(check_idx, title=f"Sample {check_idx} (Original Image)")

transform_dataset.visualize_sample(check_idx, title=f"Sample {check_idx} (Transformed Image)")
```


Sample 1 (Original Image)


Sample 1 (Transformed Image)

Check image with multiple ROIs

```
In [21]:
transform_dataset.visualize_sample(duplicate_idx, title=f"Sample with Multiple ROIs")
```


Sample with Multiple ROIs

Train test split

```
In [22]:
# Split based on patient IDs
unique_patients = paired_df['PatientID_full'].unique()
train_patients, test_patients = train_test_split(unique_patients, test_size=0.2, random_state=42)
```

```
In [23]:
train_df = paired_df[paired_df['PatientID_full'].isin(train_patients)].copy()
test_df = paired_df[paired_df['PatientID_full'].isin(test_patients)].copy()
```

```
In [24]:
train_dataset = CBISDDSM_RCNN_Dataset(dataframe=train_df, transform=rcnn_train_transform)

test_dataset = CBISDDSM_RCNN_Dataset(dataframe=test_df, transform=rcnn_test_transform)
```

```
In [25]:
def collate_fn(batch):
    return tuple(zip(*batch))
```

```
In [26]:
train_loader = DataLoader(
    train_dataset,
    batch_size=4,
    shuffle=True,
    collate_fn=collate_fn
)

test_loader = DataLoader(
    test_dataset,
    batch_size=4,
    shuffle=False,
    collate_fn=collate_fn
)
```

Initialize Faster R-CNN model and check for cuda

```
In [27]:
def get_object_detection_model(num_classes: int) -> FasterRCNN:
    """
    Creates and returns a Faster R-CNN object detection model based on a ResNet-50 backbone.

    The configured model uses the Faster R-CNN framework with a ResNet-50 backbone, which is pre-trained
    on the COCO dataset to initialize the weights. It modifies the box predictor to accommodate
    the specified number of classes by adapting the classification layer.

    :param num_classes: The number of classes including background class for the object detection model.
    :type num_classes: int
    :return: A Faster R-CNN model configured for the given number of classes.
    :rtype: FasterRCNN
    """

    # Base model
    model = torchvision.models.detection.fasterrcnn_resnet50_fpn(
        weights=torchvision.models.detection.FasterRCNN_ResNet50_FPN_Weights.DEFAULT
    )

    # Configure the model for the specified number of classes
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

    return model


num_classes = 2  # Background or abnormality
model = get_object_detection_model(num_classes)

# Use CUDA if available
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
In [28]:
# Ensure the checkpoint directory exists
checkpoint_dir = '../models/Faster_RCNN/checkpoints'
os.makedirs(checkpoint_dir, exist_ok=True)
```

```
In [29]:
# Ensure the completed model directory exists
completed_dir = '../models/Faster_RCNN/Completed'
os.makedirs(completed_dir, exist_ok=True)
```

```
In [30]:
def get_model_file_path(prompt: str, initial_directory: str, loop: bool = True) -> str | None:
    """
    Retrieve the file path of a selected model file using a graphical file dialog. Provides a simple prompt for
    the user and allows for looping interaction until a file is selected or the process is exited.

    :param prompt: Message to display to the user in the input prompt.
    :type prompt: str
    :param initial_directory: Initial directory to open in the file dialog.
    :type initial_directory: str
    :param loop: If set to True, the function will loop until the user provides valid input.
        Defaults to True.
    :type loop: bool
    :return: The file path of the selected model file, an empty string if 'n' is chosen,
        or None if the loop is ended without selection.
    :rtype: str | None
    """

    run_once = False
    while loop and not run_once:
        run_once = True

        check = input(prompt)

        if check == 'y':
            root = tk.Tk()
            root.withdraw()
            root.attributes('-topmost', True)

            return filedialog.askopenfilename(parent=root,
                                              initialdir=initial_directory,
                                              title="Select a file",
                                              filetypes=(("Model Files", "*.pth"), ("All Files", "*.*"))
                                              )
        elif check == 'n':
            return ''
    return None
```

Check for trained models

```
In [30]:
completed_path = None

if len(os.listdir(completed_dir)) > 0:
    completed_path = get_model_file_path("Completed model directory is not empty. Do you want to use saved model? (y/n): ",
                                         "../models/Faster_RCNN/Completed")

    if completed_path:
        print(f"Using saved model from {completed_path}")
    else:
        print("No model file selected. Starting training from scratch.")
```

Check for checkpoints

```
In [31]:
resume_path = None

if not completed_path and len(os.listdir(checkpoint_dir)) > 0:
    resume_path = get_model_file_path("Checkpoint directory is not empty. Do you want to resume training? (y/n): ",
                                      "../models/Faster_RCNN/checkpoints")

    if resume_path:
        print(f"Resuming training from checkpoint {resume_path}")
    else:
        print("No checkpoint file selected. Starting training from scratch.")
```

Train Faster R-CNN model

```
In [32]:
params = [p for p in model.parameters() if p.requires_grad]

optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=0.0005)

lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.1)

num_epochs = 10
start_epoch = 0

# No need for training if using an already trained model
if completed_path:
    if os.path.exists(completed_path):
        print(f"Loading model from {completed_path}")
        saved = torch.load(completed_path, map_location=device)
        model.load_state_dict(saved)
        model.eval()
        print("Model loaded successfully.")
elif resume_path:
    if os.path.exists(resume_path):
        print(f"Loading checkpoint from {resume_path}")

        # Load model from checkpoint
        checkpoint = torch.load(resume_path, map_location=device)
        model.load_state_dict(checkpoint['model_state_dict'])
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        start_epoch = checkpoint['epoch'] + 1

        print(f"Resuming training from epoch {start_epoch}")
    else:
        raise FileNotFoundError(f"Checkpoint file {resume_path} not found.")

if not completed_path:
    # Use autocast for mixed precision training (AMP) for faster training
    scaler = GradScaler()
    for epoch in range(start_epoch, num_epochs):
        model.train()
        epoch_loss = 0

        for i, (images, targets) in enumerate(train_loader):
            # Move images and targets to the appropriate device (GPU if available or CPU otherwise)
            images = list(image.to(device) for image in images)
            targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

            # Reset gradients
            optimizer.zero_grad()

            # Forward pass and loss calculation
            with autocast(device_type='cuda' if torch.cuda.is_available() else 'cpu', enabled=True):
                loss_dict = model(images, targets)
                losses = sum(loss for loss in loss_dict.values())

            # Backward pass and optimization step
            scaler.scale(losses).backward()
            scaler.step(optimizer)
            scaler.update()

            epoch_loss += losses.item()

            if i % 10 == 0:
                print(f"Epoch [{epoch + 1}/{num_epochs}] | Batch [{i}/{len(train_loader)}] | Loss: {losses.item():.4f}")

        lr_scheduler.step()
        print(f"--- Epoch {epoch + 1} Completed | Average Loss: {epoch_loss / len(train_loader):.4f} ---")

        # Save checkpoint
        checkpoint_path = os.path.join(checkpoint_dir, f'rcnn_checkpoint_{epoch + 1}.pth')
        checkpoint = {
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': epoch_loss
        }
        torch.save(checkpoint, checkpoint_path)
        print(f"Checkpoint saved to {checkpoint_path}")
```

```
Loading model from '../models/Faster_RCNN/Completed/0.6392.pth'
Model loaded successfully.
```

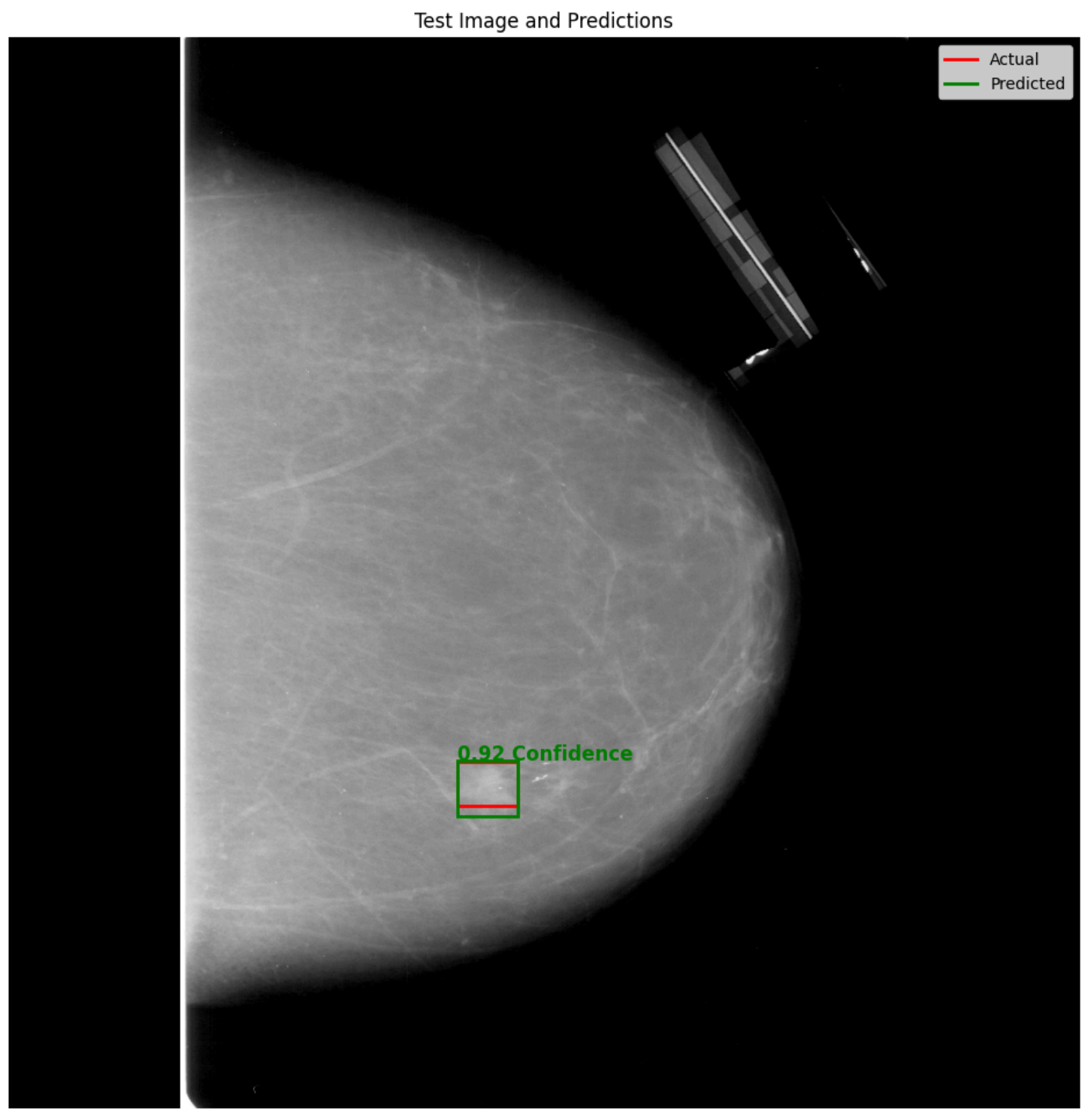
```
In [33]:
# Save the trained model only after completing training
if not completed_path:
    model_save_path = '../models/Faster_RCNN/Completed/Completed_Training.pth'
    torch.save(model.state_dict(), model_save_path)
    print(f"Model weights saved to {model_save_path}")
```

```
Model weights saved to ../models/Faster_RCNN/Completed/Completed_Training.pth
```

Visualize predicted image

```
In [34]:
def test_and_visualize(model: FasterRCNN, test_loader: DataLoader, device: torch.device, confidence_threshold: float=0.2, idx: int=0) -> None:
    """
    Test and visualize the performance of a Faster R-CNN model on a single batch of test images. The method generates
    a visualization that overlays both true bounding boxes and predicted bounding boxes onto the test image. The color
    red is used for true boxes, while green represents predicted boxes with a confidence score higher than the specified
    threshold. Predictions are annotated with their confidence scores.

    :param model: Faster R-CNN model
    :param test_loader: DataLoader object supplying a batch of test images and their corresponding ground truths
    :param device: PyTorch device specifying where the tensors should be allocated (CPU or CUDA device)
    :param confidence_threshold: Float value indicating the minimum confidence score for visualizing predicted bounding boxes
    :param idx: Index of the image in the batch to visualize
    :return: None
    """

    model.eval()

    images, targets = next(iter(test_loader))
    images = list(img.to(device) for img in images)

    # Predict bounding boxes for batch
    with torch.no_grad():
        predictions = model(images)

    # Convert image tensors to readable numpy arrays and ensure valid pixel values
    img_np = images[idx].cpu().permute(1, 2, 0).numpy()
    img_np = np.clip(img_np, 0, 1)

    fig, ax = plt.subplots(1, figsize=(12, 12))

    if img_np.shape[-1] == 1:
        ax.imshow(img_np.squeeze(-1), cmap='gray')
    else:
        ax.imshow(img_np)

    # True boxes (Red)
    true_boxes = targets[idx]['boxes'].cpu().numpy()

    for box in true_boxes:
        xmin, ymin, xmax, ymax = box
        rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin, linewidth=2, edgecolor='r', facecolor='none')
        ax.add_patch(rect)

    # Predicted boxes (Green)
    predicted_boxes = predictions[idx]['boxes'].cpu().numpy()
    predicted_scores = predictions[idx]['scores'].cpu().numpy()

    for box, score in zip(predicted_boxes, predicted_scores):
        if score >= confidence_threshold:
            xmin, ymin, xmax, ymax = box
            rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin, linewidth=2, edgecolor='g',
                                     facecolor='none')
            ax.add_patch(rect)
            plt.text(xmin, ymin, f"{score:.2f} Confidence", color='g', fontsize=12, weight='bold')
    plt.axis('off')
    plt.title("Test Image and Predictions")

    legend_elements = [Line2D([0], [0], color='r', lw=2, label='Actual'),
                       Line2D([0], [0], color='g', lw=2, label='Predicted')]
    ax.legend(handles=legend_elements, loc='upper right')

    plt.show()
```

```
In [35]:
test_and_visualize(model, test_loader, device, idx=0)
```


Test Image and Predictions

Calculate mAP

```
In [36]:
def calculate_test_map(model, test_loader, device):
    print("Calculating test set mAP...")
    metric = MeanAveragePrecision(box_format='xyxy', class_metrics=True)

    model.eval()

    print(f"Evaluating over {len(test_loader)} batches.")
    with torch.no_grad():
        for imgs, targets in tqdm(test_loader):
            imgs = list(img.to(device) for img in imgs)

            predictions = model(imgs)

            formatted_predictions = []
            for p in predictions:
                formatted_predictions.append(dict(
                    boxes=p['boxes'].cpu(),
                    scores=p['scores'].cpu(),
                    labels=p['labels'].cpu()
                ))

            formatted_targets = []
            for t in targets:
                formatted_targets.append(dict(
                    boxes=t['boxes'].cpu(),
                    labels=t['labels'].cpu()
                ))

            metric.update(formatted_predictions, formatted_targets)

    print("Computing final mAP...")
    results = metric.compute()
    print("Done")

    return results
```

```
In [37]:
map_results = calculate_test_map(model, test_loader, device)
```

```
Calculating test set mAP...
Evaluating over 76 batches.
100%|██████████| 76/76 [01:33<00:00,  1.23s/it]
Computing final mAP...
Done
```

```
In [38]:
print(f"mAP @ IoU=0.50:0.95 (COCO Standard): {map_results['map'].item():.4f}")
print(f"mAP @ IoU=0.50 (PASCAL VOC Standard): {map_results['map_50'].item():.4f}")
```

```
mAP @ IoU=0.50:0.95 (COCO Standard): 0.2731
mAP @ IoU=0.50 (PASCAL VOC Standard): 0.6392
```